University Of Information Technology & Sciences

PROJECT REPORT

Course Title : Linux Programming Lab
Course Code : CSE0611304
Project Name : Simple Password Manager with Secure Search & GUI Integration

Submitted To:

Md. Ismail, Lecturer, CSE &
Md. Tasnin Tanvir, Lecturer, CSE

Submitted By:

Mumtahina Maria
Id: 0432410005101064 (5B1)

Shalehin Ahmed Ornob
Id: 0432410005101183 (5B1)

Submission Date: 19 May, 2026

Github Repository Link: SecurePass

1. Introduction

Password security remains one of the most critical aspects of modern digital life. With the increasing number of online accounts and services, users often struggle to maintain unique, strong passwords for each platform. Weak or reused passwords are among the leading causes of data breaches and unauthorized access across the globe.

This project — SecurePass: Simple Password Manager with Secure Search and GUI Integration — was developed as part of the Linux Programming Lab curriculum to demonstrate the real-world application of shell scripting combined with a modern desktop graphical interface. The system enables users to generate cryptographically random passwords, store them locally, search for specific entries, and delete outdated records — all through an intuitive GUI powered by Electron.js.

The backend is entirely built on Bash Shell Scripting, leveraging Linux's built-in utilities such as /dev/urandom, grep, cat, tr, and head. The frontend uses Electron.js, HTML5, Tailwind CSS, and DaisyUI to provide a clean, responsive desktop interface. Inter-Process Communication (IPC) bridges the two layers seamlessly.

2. Project Objectives

- Generate cryptographically strong random passwords of configurable length
- Store generated passwords with their corresponding website or account labels in a text file
- View all saved password records in a readable format
- Search for specific password entries using case-insensitive pattern matching
- Delete unwanted or outdated password records safely
- Integrate Bash shell scripting with a modern Electron.js desktop GUI
- Make password management accessible to non-technical users through a GUI

3. Project Description

SecurePass is a desktop-based password management application developed as part of a Linux Programming Lab project. It combines the power of Bash shell scripting as the backend engine with a modern Electron.js GUI frontend, making password management both technically robust and user-friendly.

The core backend is a modular shell script (manager.sh) that handles four key operations — generating cryptographically random passwords using /dev/urandom, storing them in a local file in site:password format, searching entries with case-insensitive grep, and safely deleting records using a temporary file replacement technique.

The frontend is an Electron.js desktop application styled with Tailwind CSS and DaisyUI, featuring a clean card-based interface for each operation. Communication between the GUI and the shell script is handled through Electron's IPC (Inter-Process Communication) system — ipcMain in the main process invokes the shell script via child_process.exec(), while contextBridge in the preload script securely exposes the API to the renderer.

The project demonstrates practical integration of Linux utilities, shell scripting fundamentals, and modern desktop application development — making it fully usable both from the command line and through the graphical interface.

4. Tools and Technologies Used

Category	Technology / Tool	Purpose
Backend	Bash Shell Script	Core password management logic
Backend	/dev/urandom	Cryptographically secure random source
Backend	tr, head, grep, cat	Linux command-line utilities
Backend	passwords.txt	Local file-based storage
Frontend	Electron.js v42	Desktop application framework
Frontend	HTML5	Interface structure and layout
Frontend	Tailwind CSS	Utility-first responsive styling
Frontend	DaisyUI	Pre-built component library
Frontend	JavaScript (ES6)	Client-side event handling
Bridge	ipcMain / ipcRenderer	Main-Renderer process communication
Bridge	contextBridge	Secure API exposure to renderer
Dev Tools	VS Code	Code editor
Dev Tools	Node.js + npm	JavaScript runtime and package manager
Dev Tools	Git Bash / WSL	Linux shell on Windows

5. Backend, Frontend Implementation and Others

5.1. Bbackend Implementation

The backend is a single Bash script that handles all four password management operations. It acts as the core engine of the application and can be used independently from the command line.

5.1.1 Password Generation

The generate_password() function uses /dev/urandom — a kernel-provided source of cryptographically secure pseudorandom bytes — piped through tr to filter only printable characters, and head to truncate at the desired length. The result is appended to passwords.txt in site:password format.

5.1.2 View All Passwords

The view_passwords() function checks whether passwords.txt is non-empty using the -s flag. If records exist, it uses cat to display all entries. Otherwise, it shows a friendly message.

5.1.3 Search Password

The search_password() function uses grep with the -i flag for case-insensitive matching. If no entry is found, the || operator ensures a fallback message is displayed.

5.1.4 Delete Password

The delete_password() function uses grep -iv to invert the match — filtering out any line beginning with the target site name followed by a colon. The filtered output is redirected to a temporary file, which then replaces the original passwords.txt using mv.

5.1.5 Full Backend Code (manager.sh)

```
#!/bin/bash

PASSWORD_FILE="passwords.txt"

generate_password() {
    length=$1
    site=$2

    password=$(tr -dc 'A-Za-z0-9!@#$%^&*()_+=' < /dev/urandom | head -c $length)

    echo "$site:$password" >> $PASSWORD_FILE
```

```
    echo "Generated Password for $site: $password"
}

view_passwords() {
    if [ -s $PASSWORD_FILE ]; then
        cat $PASSWORD_FILE
    else
        echo "No passwords saved yet."
    fi
}

search_password() {
    search=$1
    grep -i "$search" $PASSWORD_FILE || echo "No matching entry found."
}

delete_password() {
    delete=$1
    grep -iv "^$delete:" $PASSWORD_FILE > temp.txt
    mv temp.txt $PASSWORD_FILE
    echo "Entry deleted if it existed."
}

case $1 in
    generate)
        generate_password $2 $3
        ;;
    view)
        view_passwords
        ;;
    search)
        search_password $2
        ;;
    delete)
        delete_password $2
        ;;
    *)
        echo "Usage:"
        echo "./manager.sh generate <length> <site>"
        echo "./manager.sh view"
        echo "./manager.sh search <site>"
        echo "./manager.sh delete <site>"
        ;;
esac
```

5.1.6 Command-Line Usage

```
# Generate a 16-character password for GitHub
./manager.sh generate 16 github

# View all saved passwords
./manager.sh view
```

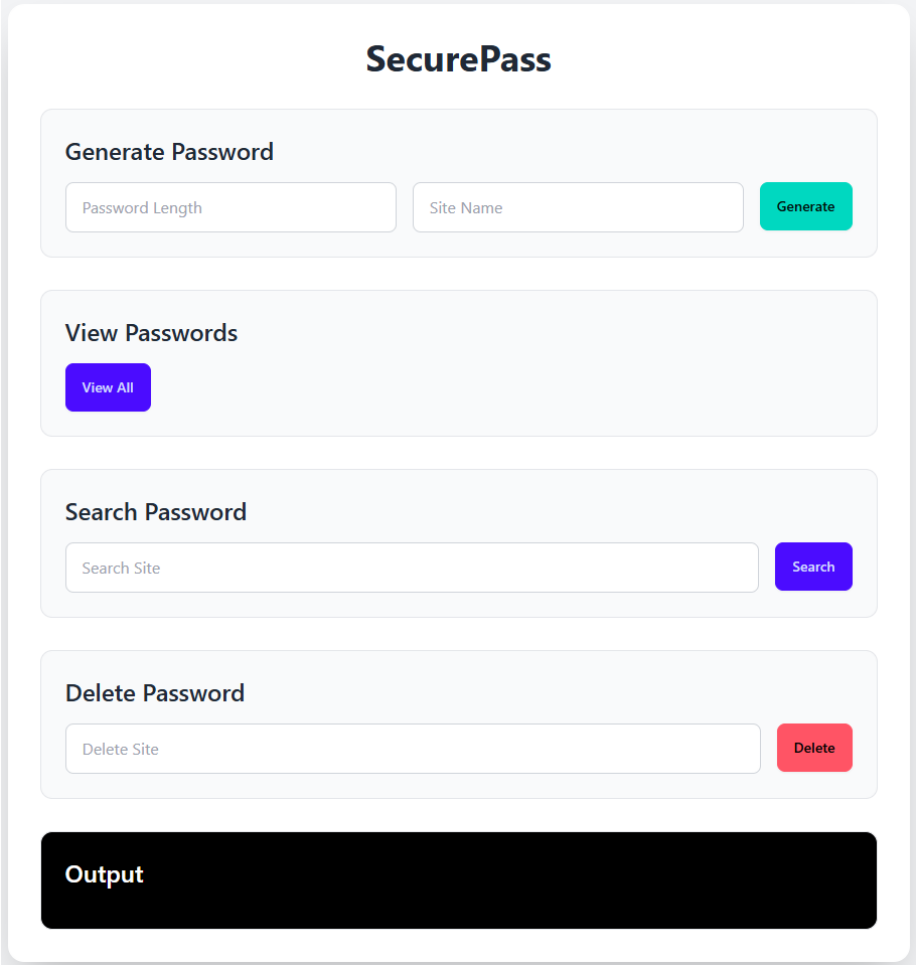
```
# Search for a specific site
./manager.sh search github

# Delete an entry
./manager.sh delete github
```

5.2 Frontend Implementation — Electron GUI

The frontend consists of three layers: the HTML interface (index.html), the renderer script (renderer.js), and the Electron preload bridge (preload.js). Together they create a responsive, modern desktop GUI.

5.2.1 HTML Interface (index.html)

The interface is structured with four functional cards — Generate, View, Search, and Delete — plus a terminal-style output area. Tailwind CSS handles responsive layout and DaisyUI provides styled button and input components.



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SecurePass GUI</title>
  <link href="https://cdn.jsdelivr.net/npm/daisyui@latest/dist/full.css"
        rel="stylesheet" type="text/css" />
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 min-h-screen flex items-center justify-center p-6">
  <div class="bg-white shadow-2xl rounded-2xl w-full max-w-4xl p-8 space-y-8">
    <h1 class="text-4xl font-bold text-center text-gray-800">SecurePass</h1>
    <!-- Generate Section -->
    <div class="border rounded-xl p-6 bg-gray-50">
      <h2 class="text-2xl font-semibold mb-4">Generate Password</h2>
      <input type="number" id="length" placeholder="Password Length" class="input-
field"/>
      <input type="text"   id="site"   placeholder="Site Name"        class="input-
field"/>
      <button id="generateBtn" class="btn btn-accent">Generate</button>
    </div>
    <!-- Output Terminal -->
    <div class="border rounded-xl p-6 bg-black text-green-400">
      <h2 class="text-2xl font-semibold mb-4 text-white">Output</h2>
      <pre id="output" class="whitespace-pre-wrap"></pre>
    </div>
  </div>
  <script src="renderer.js"></script>
</body>
</html>
```

8.2.2 Renderer Script (renderer.js)

renderer.js wires each button to its corresponding Electron API call. Async/await is used for all IPC calls, with try/catch blocks to handle and display errors in the output terminal.

```
const output = document.getElementById("output");

// Generate Password
document.getElementById("generateBtn").addEventListener("click", async () => {
    const length = document.getElementById("length").value;
    const site = document.getElementById("site").value;
    try {
        const result = await window.electronAPI.generatePassword(length, site);
        output.textContent = result;
    } catch (error) {
        output.textContent = "Error: " + error;
    }
});
```

```
// View Passwords
document.getElementById("viewBtn").addEventListener("click", async () => {
    try {
        const result = await window.electronAPI.viewPasswords();
        output.textContent = result;
    } catch (error) {
        output.textContent = "Error: " + error;
    }
});

// Search Password
document.getElementById("searchBtn").addEventListener("click", async () => {
    const site = document.getElementById("searchSite").value;
    try {
        const result = await window.electronAPI.searchPassword(site);
        output.textContent = result;
    } catch (error) {
        output.textContent = "Error: " + error;
    }
});

// Delete Password
document.getElementById("deleteBtn").addEventListener("click", async () => {
    const site = document.getElementById("deleteSite").value;
    try {
        const result = await window.electronAPI.deletePassword(site);
        output.textContent = result;
    } catch (error) {
        output.textContent = "Error: " + error;
    }
});
```

5.3 IPC Communication Bridge

Electron's Inter-Process Communication (IPC) system is the backbone of the GUI-to-script integration. It consists of two files:

5.3.1 main.js — IPC Handlers

main.js runs in the main process and registers ipcMain.handle() listeners for each operation. Each handler invokes the shell script using Node.js's child_process.exec(), captures stdout, and resolves the promise returned to the renderer.

```
const { app, BrowserWindow, ipcMain } = require("electron");

const path = require("path");
const { exec } = require("child_process");
```

```
function createWindow() {
    const win = new BrowserWindow({
        width: 1000,
        height: 800,
        webPreferences: {
            preload: path.join(__dirname, "preload.js")
        }
    });
    win.loadFile("index.html");
}

app.whenReady().then(() => { createWindow(); });
const scriptPath = path.join(__dirname, "../shell-scripting/manager.sh");

// Generate Password
ipcMain.handle("generate-password", async (event, length, site) => {
    return new Promise((resolve, reject) => {
        exec(`bash "${scriptPath}" generate ${length} ${site}`,
            (error, stdout, stderr) => {
                if (error) reject(stderr); else resolve(stdout);
        });
    });
});

// View Passwords
ipcMain.handle("view-passwords", async () => {
    return new Promise((resolve, reject) => {
        exec(`bash "${scriptPath}" view`,
            (error, stdout, stderr) => {
                if (error) reject(stderr); else resolve(stdout);
        });
    });
});

// Search Password
ipcMain.handle("search-password", async (event, site) => {
    return new Promise((resolve, reject) => {
        exec(`bash "${scriptPath}" search ${site}`,
            (error, stdout, stderr) => {
                if (error) reject(stderr); else resolve(stdout);
        });
    });
});

// Delete Password
ipcMain.handle("delete-password", async (event, site) => {
    return new Promise((resolve, reject) => {
        exec(`bash "${scriptPath}" delete ${site}`,
            (error, stdout, stderr) => {
                if (error) reject(stderr); else resolve(stdout);
        });
    });
});
```

5.3.2 preload.js — Context Bridge

preload.js uses Electron's contextBridge to safely expose the IPC methods to the renderer process. This follows Electron's security best practices by avoiding direct exposure of Node.js APIs to the front-end JavaScript context.

```
const { contextBridge, ipcRenderer } = require("electron");

contextBridge.exposeInMainWorld("electronAPI", {
    generatePassword: (length, site) =>
        ipcRenderer.invoke("generate-password", length, site),
    viewPasswords: () =>
        ipcRenderer.invoke("view-passwords"),
    searchPassword: (site) =>
        ipcRenderer.invoke("search-password", site),
    deletePassword: (site) =>
        ipcRenderer.invoke("delete-password", site)
});
```

5.4 Output (generate, show all, search, delete)

Output

```
Generated Password for Shakil: UA$qpTlD9adG
```

Output

```
SA:O)BDiVOl8uy$
Maria:h6So^adc^88j
Joy:=lyF@^ksVni6
```

Output

```
Shakil:UA$qpTlD9adG
```

Output

```
Entry deleted if it existed.
```

6. Future Improvements

- Encrypt passwords.txt using GPG or AES-256 before storage
- Add a master password / PIN requirement to launch the application
- Implement a 'Copy to Clipboard' button in the GUI for convenience
- Add password strength meter and enforce minimum complexity rules
- Migrate storage from flat-file to SQLite for better scalability and search performance
- Sanitize and validate all user inputs to prevent shell injection attacks
- Add password categories and tags for better organization
- Implement auto-clear clipboard after 30 seconds for security
- Package the Electron app as a distributable installer (.exe / .deb / .dmg)
- Add import/export functionality for backup and portability

7. Conclusion

This project successfully demonstrates the integration of Bash shell scripting with a modern desktop GUI using Electron.js. The SecurePass Password Manager covers all fundamental operations of a password management system — generation, storage, retrieval, search, and deletion — implemented entirely through Linux shell utilities.

The project bridges the gap between command-line tools and user-friendly interfaces, making shell scripting concepts accessible to a wider audience. It also highlights the power and flexibility of Linux's built-in tools, such as /dev/urandom, grep, tr, and file redirection, in building real-world applications.

From a learning perspective, this project reinforces key concepts in Linux programming including process communication, file I/O, string manipulation, function-based scripting, and cross-platform desktop development. It serves as a solid foundation for more advanced projects involving encryption, database integration, and system security.

8. References

1. GNU Bash Reference Manual — https://www.gnu.org/software/bash/manual/
2. Electron.js Official Documentation — https://www.electronjs.org/docs/latest
3. Tailwind CSS Documentation — https://tailwindcss.com/docs
4. DaisyUI Component Library — https://daisyui.com/
5. Node.js child process Documentation — https://nodejs.org/api/child_process.html